CSCI 135 Assignment – Lists

September 27th / 29th, 2023

Due Date: Sunday, 10/1/2023 11:59PM

Points: 40

Topic: Introduction to Lists

The goal of this assignment is to gain experience using lists. Continue making your code organized and readable. Pay attention to your header comments and include comments in-line with your code. Use proper indentation and understandable variable names. Use whitespace to help with readability and logical grouping of sections of code.

Submission:

After following the instructions below, please submit the two Python scripts in Moodle. Upload these as separate files – do not zip them. Be sure each file has a header section including the file name, your name, a description of the program, and other useful information such as example usage.

- Matrix.py
- Password.py

Grading:

Each program/script is worth 20 points each for a total of 40 points. For each program, you will be graded according to the following criteria:

	Points
Header Comment	4
Program Written Correctly, Following Direction	8
Program Runs Correctly, Including Correct Output	8
Total	**20**

Part 1: Password.py (aka, Password Challenge-Response)

Write a program named Password.py that does the following.

Traditional password entry schemes are susceptible to "shoulder surfing" in which an attacker watches an unsuspecting user enter his or her password or PIN number and uses it later to gain access to the user's account. One way to combat this problem is with a randomized challenge-response system. In these systems the user enters different information every time based on a secret in response to a randomly generated challenge.

Consider the following process:

- A user's actual password is a five-digit PIN. I.e., it falls between 00000 and 99999, inclusive.
- Per each logon attempt, the possible digits are each assigned a random number: 1, 2 or 3.
- Instead of entering their PIN, users enter the random numbers corresponding to their PIN.

For example, consider an actual PIN of 12345. To authenticate, the user would be presented with a screen such as the following and respond as shown:

```
PIN: 0 1 2 3 4 5 6 7 8 9
NUM: 3 2 3 1 1 3 2 2 1 3

Please enter your converted password: 23113

Welcome! You are correct.
```

The user's entry of 23113 is correct because it matches, in order, the random numbers that correspond to the user's PIN digits. This approach doesn't divulge the password even if an attacker intercepts the entry because the converted password 23113 corresponds to other PIN numbers such as 69440 or 70439.

The next time the user logs in, a different sequence of random numbers is generated:

```
PIN: 0 1 2 3 4 5 6 7 8 9
NUM: 1 1 2 3 1 2 2 3 3 3

Please enter your converted password: 12312

You are correct. Welcome!
```

Write a program to simulate this authentication process and produce output as shown:

- Store (hardcode) the actual PIN in your program. For this program, use the "actual" PIN number of 56789. Store this in a list of integers (int) in your program as follows:

  ```
  pin = [5, 6, 7, 8, 9]
  ```

- Your program shall use a list to assign the random numbers (1, 2 or 3) to the digits from 0 to 9. This will be a list of integers (int) where the index is the digit and value is the corresponding random 1, 2 or 3. Use list comprehension to do this all in one line, as follows. You will need to import of the random package prior to this statement:

  ```
  randNumbers = [random.randint(1,3) for x in range(10)]
  ```

- Output the random digits and their correspondence to actual digits to the screen as shown in the examples above and below.

- Request and get the input response from the user using a single input() with the prompt shown. Your input will come in as a single string / text.

- Output whether or not the user's response correctly matches the PIN number as shown.

Here are several more example runs where the actual (hard-coded) PIN is 56789.

In this run, the user entered an incorrect converted password:

```
PIN: 0 1 2 3 4 5 6 7 8 9
NUM: 2 1 2 3 3 2 3 1 3 1

Please enter your converted password: 23133

Sorry, that password is incorrect.
```

In this run, the user entered the correct converted password.

```
PIN: 0 1 2 3 4 5 6 7 8 9
NUM: 3 1 3 2 1 1 2 2 3 3

Please enter your converted password: 12233

Welcome! You are correct.
```

Helpful hints:

How do I get the individual digits from the user's input?

You are to use a single input() call. Input will come in as a single text string. You can extract the integer values of the input by extracting each character and converting it to an integer. Example:

If pwdEntry = "13221", then pwdEntry[1] is '3' and int(pwdEntry[1]) is 3.

```
>>>
>>> pwd = "13221"
>>>
>>> pwd[1]
'3'
>>>
>>> int(pwd[1])
3
>>>
```

How do I get items to output to the same line?

Example:

```
print("PIN: ", end="")
```

The end="" parameter will cause the next item printed via a print() call to be printed on the same line. You will need to use a separate print() statement without the end parameter or use a "\n" character to end a line.

How do I get a list of random integers between 1 and 3?

You will need to import the random package, and then use the random.randint() function, as shown above and as follows:

```
import random
 ...
randNumbers = [random.randint(1,3) for x in range(10)]
```

This code generates the list of random numbers all in one list comprehension similar to the following example:

```
>>> import random
>>> [random.randint(1,3) for x in range(10)]
[2, 1, 3, 1, 2, 2, 1, 2, 1, 2]
```

How do I get started?

There is a lot to this program. Here are some suggestions:

- Read the problem carefully and write down what steps you need to do. Make note of the steps where I have given you code.
- Make sure you use the code I have given you.
- Define the list that the "actual" PIN is stored in. I give you code above that does this.
- Define the list that stores the random digits from 1 to 3 that are associated with each actual digit so that you can display it to the user. I give you code above that does this.
- Get the user input. Follow the specifications above to do this.
- Check to see if each digit matches the correct entry in the random number list. I show you above how to extract an individual digit from the input string and convert it to an integer. You need to figure out how to use this to compare the whole input to the correct random numbers. You may want to use a Boolean variable to help accomplish this.
- Work incrementally. Write code to do pieces of the problem in order and test the pieces as you go. Print out intermediate results and make sure they are correct before moving on.
- Try things in the Python interactive console to see that they work. See screenshots above.

Upload Password.py to Moodle as the first part of your submission for this assignment.

Part 2: Enter the Matrix

Write a program named Matrix.py that does the following.

Create a list of lists to represent a 5 x 5 numeric matrix, initialized to all zeros. Print the matrix. Use nested for loops to iterate through the matrix elements. See below for example output.

Using nested for loops, set the value of each non-diagonal matrix element to a random floating-point number between 0 and 150. Use the following to generate each number:

```
150 * random.random()
```

Print the matrix. See below for example (messy) output.

Print the matrix again with improved formatting. Use an f-string to print each value so it is aligned on the right, rounded to 2 places, and occupies no more than 8 characters including separating / padding spaces.

Example Output:

```
All Zeros:

0  0  0  0  0
0  0  0  0  0
0  0  0  0  0
0  0  0  0  0
0  0  0  0  0

Messy Output:

0  109.68587997971808  19.611219499061622  139.04947120633523  24.858557835262644
1.3929374199159184  0  23.45279627269799  0.0859552695726351  36.37384140521699
34.22888666945251  29.4168098752632  0  59.4939367728059  122.46109088515612
130.54545618837125  27.186392685249412  20.080455567386668  0  33.788673223329845
31.997848133823975  134.35815734507713  36.750152796484564  128.79529273877102  0


Clean Output:

  0.00  109.69   19.61  139.05   24.86
  1.39    0.00   23.45    0.09   36.37
 34.23   29.42    0.00   59.49  122.46
130.55   27.19   20.08    0.00   33.79
 32.00  134.36   36.75  128.80    0.00
```

Helpful hints:

How do I generate random floating-point numbers in the given range?

Use the code that was given above:

```
150 * random.random()
```

Don't forget to import the random package.

How do I create a list of lists and access an element in the list.

Here is a simple example:

```
theListOfLists = [ [1 , 2] , [3 , 4] ]
```

Then `print(theListOfLists[0][1])` would print the number `2`. Make sure you understand what the indexes reference here.

How do I use an f-string to get the desired formatting:

Try this:

```
f"{val:>8.2f}"
```

You may need to use the end parameter in your print statements as shown in the prior exercise.

Upload Matrix.py to Moodle as the first part of your submission for this assignment.

EXTENSIONS

You will not submit the answers to these extensions as part of your assignment submission. You should work through and understand them to learn more and practice. You may be held responsible for the content within, so you really should work through them …

From the Liang book - NOTE: We will **not** use MyProgrammingLab in the book. You should be able to do exercises from Liang without MyProgrammingLab.

1. *Work through the case study in Section 10.3. Note how this goes beyond what was done in our prior Lottery assignment, and the constructs used to give new capability.*

2. *Work through the case study in Section 10.4. How are suits and ranks of cards handled in this example? What would it take to make a simple, console-based card game using this representation of cards? Do we have the tools to do this?*

3. *Read Section 10.6 carefully. How does this jive with our prior understanding of variables and objects?*

4. *Try exercises 10.1 – 10.7.*

5. *Work through other exercises and examples related to lists in Sections 10.1 – 10.4 and 10.6. Do the easy ones first. Try some of the more challenging ones.*

6. *Work through the Guessing Birthdays problem 11.9.2. Compare this to the code from Listing 4.3. (Note the use of a main() function here. We will discuss this later in the semester.)*

7. *Try exercises 11.1 and 11.2.*

8. *Work through other exercises and examples related to multidimensional-lists in Sections 11.1, 11.2 and 11.9. Do the easy ones first. Try some of the more challenging ones.*

9. *Work through the comparison of lists and sets in Section 14.4. Why are sets faster than lists here?*

10. *Work through other exercises and examples related to tuples and sets in Sections 14.1, 14.2, 14.3 and 14.4. Do the easy ones first. Try some of the more challenging ones.*

From the *Things to Do* section at the end of Chapter Seven and Chapter Eight of our optional Lubanovic book:

7.1 Create a list called `years_list`, starting with the year of your birth, and each year thereafter until the year of your fifth birthday. For example, if you were born in 1980, the list would be `years_list = [1980, 1981, 1982, 1983, 1984, 1985]`. If you're less than five years old and reading this book, I don't know what to tell you.

7.2 In which year in `years_list` was your third birthday? Remember, you were 0 years of age for your first year.

7.3 In which year in `years_list` were you the oldest?

7.4 Make a list called `things` with these three strings as elements: `"mozzarella"`, `"cinderella"`, `"salmonella"`.

7.5 Capitalize the element in `things` that refers to a person and then print the list. Did it change the element in the list?

7.6 Make the cheesy element of `things` all uppercase and then print the list.

7.7 Delete the disease element from `things`, collect your Nobel Prize, and print the list.

7.8 Create a list called `surprise` with the elements `"Groucho"`, `"Chico"`, and `"Harpo"`.

7.9 Lowercase the last element of the `surprise` list, reverse it, and then capitalize it.

7.10 Use a list comprehension to make a list called `even` of the even numbers in `range(10)`.

7.11 Let's create a jump rope rhyme maker. You'll print a series of two-line rhymes. Start with this program fragment:

```
start1 = ["fee", "fie", "foe"]
rhymes = [
    ("flop", "get a mop"),
    ("fope", "turn the rope"),
    ("fa", "get your ma"),
    ("fudge", "call the judge"),
    ("fat", "pet the cat"),
    ("fog", "walk the dog"),
    ("fun", "say we're done"),
    ]
start2 = "Someone better"
```

For each tuple (`first`, `second`) in `rhymes`:

For the first line:

- Print each string in `start1`, capitalized and followed by an exclamation point and a space.
- Print `first`, also capitalized and followed by an exclamation point.

For the second line:

- Print `start2` and a space.
- Print `second` and a period.

8.1 Make an English-to-French dictionary called `e2f` and print it. Here are your starter words: `dog` is `chien`, `cat` is `chat`, and `walrus` is `morse`.

8.2 Using your three-word dictionary `e2f`, print the French word for `walrus`.

8.3 Make a French-to-English dictionary called `f2e` from `e2f`. Use the `items` method.

8.4 Print the English equivalent of the French word `chien`.

8.5 Print the set of English words from `e2f`.

8.6 Make a multilevel dictionary called `life`. Use these strings for the topmost keys: `'animals'`, `'plants'`, and `'other'`. Make the `'animals'` key refer to another dictionary with the keys `'cats'`, `'octopi'`, and `'emus'`. Make the `'cats'` key refer to a list of strings with the values `'Henri'`, `'Grumpy'`, and `'Lucy'`. Make all the other keys refer to empty dictionaries.

8.7 Print the top-level keys of `life`.

8.8 Print the keys for `life['animals']`.

8.9 Print the values for `life['animals']['cats']`.

8.10 Use a dictionary comprehension to create the dictionary `squares`. Use `range(10)` to return the keys, and use the square of each key as its value.

8.11 Use a set comprehension to create the set `odd` from the odd numbers in `range(10)`.

8.12 Use a generator comprehension to return the string `'Got '` and a number for the numbers in `range(10)`. Iterate through this by using a `for` loop.

8.13 Use `zip()` to make a dictionary from the key tuple `('optimist', 'pessimist', 'troll')` and the values tuple `('The glass is half full', 'The glass is half empty', 'How did you get a glass?')`.

8.14 Use `zip()` to make a dictionary called `movies` that pairs these lists: `titles = ['Creature of Habit', 'Crewel Fate', 'Sharks On a Plane']` and `plots = ['A nun turns into a monster', 'A haunted yarn shop', 'Check your exits']`

CSCI 135 Assignment – Lists

From me: - Looking ahead to dictionaries …

1. How are dictionaries different from lists? How are they similar?

2. Could a dictionary be used to represent a list? Why or why not?

3. Could a list be used to represent a dictionary? Why or why not?